# Algoritmos e Estruturas de Dados
## Trabalho Prático: Árvores no Mundo Real (BST e AVL)

## 🎯 1. Objetivo do Trabalho

Integrar e aplicar, em um problema **real e verificável**, os conceitos de **Árvore Binária de Busca (BST)** e **Árvore AVL** estudados no Capítulo 9 (Trees), usando **ponteiros, structs e alocação dinâmica** em Linguagem C.

Sua tarefa é **encontrar** uma situação real em que organizar dados em árvore faz diferença, **explicar** por que ela é a estrutura adequada, **implementar** a solução e **defendê-la em vídeo**, demonstrando tudo funcionando e comparando o desempenho da BST com o da AVL.

## 🔎 2. Etapa 1: Proposta do Problema (antes de programar)

Apresentar na aula dia **14/10/26** uma proposta de **1 página** (PDF – favor enviar no moodle) contendo:

- **Contexto real:** onde o problema existe (loja, clínica, escola, empresa, aplicativo, jogo, sistema público etc.) e **como você sabe disso**. É obrigatório citar a fonte: conversa com uma pessoa que vive o problema (informe nome ou função e a data), observação direta no local ou documento/site real (com link,...), ....
- **O problema em uma frase** e quais operações precisam ser rápidas (buscar, inserir, remover, listar em ordem, consultar faixa de valores etc.).
- **A chave de ordenação** escolhida (o que é comparado para posicionar cada nó) e o **volume de dados** esperado.
- **Dados reais:** no mínimo **30 registros reais ou realisticamente coletados por você**, anexados como arquivo .txt ou .csv.

**Regra do tema único:** não pode haver dois alunos com o mesmo problema. O professor aprova a proposta (ou pede ajustes) e registra o tema em uma planilha por ordem de chegada. **Sem proposta aprovada, o trabalho final não é corrigido.**

**Atenção:** O problema precisa ter um motivo concreto para o uso de árvore.

### Pontos de partida (apenas para inspirar, não copie)

- Controle de estoque de uma loja pesquisado por código de produto.
- Consulta de agendamentos de um consultório por horário, incluindo busca por intervalo.
- Ranking de pontuação de jogadores de um campeonato do qual você participa.
- Catálogo de livros de uma biblioteca (da escola, da igreja, da cidade).
- Registro de ocorrências ou protocolos ordenados por número ou data.

Esses exemplos são apenas pontos de partida. Procure algo da sua vida, do seu trabalho ou da sua comunidade.

## 💻 3. O Desafio Técnico

### 3.1 Requisitos obrigatórios do código

- **Duas árvores sobre os mesmos dados:** implementar a **BST** e a **AVL** (com as rotações simples e duplas), ambas com nós alocados dinamicamente.
- **Operações mínimas em ambas:** inserir, buscar, remover (incluindo o caso de nó com dois filhos), percurso em ordem, pré-ordem e pós-ordem, e cálculo de altura.
- **Carga dos dados reais:** o programa deve ler o arquivo da Etapa 1 e construir as árvores a partir dele.

  **Ponteiros:** toda a navegação e a modificação da árvore devem ser feitas com ponteiros e a notação.

- **Funções:** o programa não pode ser desenvolvido no main. Cada operação deve ser uma função com parâmetros, chamada pelo main.
- **Sem variáveis globais.** Toda a passagem de dados deve ocorrer por parâmetros.
- **Verificação de alocação:** todo malloc/calloc deve ter o retorno testado contra NULL.
- **Liberação de memória:** ao encerrar, liberar **todos os nós das duas árvores** com free. Também devem ser liberados os nós "removidos" durante a execução.
- **Menu interativo** em modo texto para o professor testar todas as operações.
- **Bibliotecas:** apenas a biblioteca padrão de C (stdio, stdlib, string, time, ...). Não é permitido usar bibliotecas p/ lidar com estruturas de dados.

### 3.3 Experimento comparativo (obrigatório)

**Para que serve:** provar numericamente, usando os seus dados reais (Etapa 1), que a AVL melhora em relação à BST. O programa deve ter uma opção no menu chamada "Experimento comparativo" que executa os passos abaixo sozinha e imprime o resultado, montando a tela.

### Como contar comparações

A cada vez que a função de busca compara a chave procurada com a chave de um nó (seja para testar igualdade, menor ou maior), ela soma **1** em um contador. Esse contador é uma variável criada na função que chama a busca e **passada por ponteiro** para a função de busca (sem variável global). Para cada árvore, zere o contador antes de começar mostre/documente o valor.

### Passo a passo do experimento

1. **Cenário A (ordem original):** leia os registros do arquivo na ordem em que aparecem e insira cada um em uma BST nova e em uma AVL nova. Anote a **altura** de cada árvore.
2. Para **cada registro** do arquivo, busque na BST usando sua chave (e depois na AVL), acumulando as comparações em um contador para cada árvore. Ao final, calcule a **média de comparações por busca** (total de comparações dividido pelo número de registros).
3. **Cenário B (ordem crescente):** ordene os registros por chave, construa **duas árvores novas** inserindo nessa ordem e repita o passo 2.
4. Libere a memória das quatro árvores do experimento e mostre a tabela final.

### Tabela que o programa deve imprimir

Uma linha por combinação de cenário e árvore, com altura, total de comparações e média por busca. O exemplo abaixo mostra apenas o **formato** e usa N chaves inseridas em ordem crescente, para ilustrar o Cenário B.

| Cenário | Árvore | Altura | Comparações totais | Média por busca |
|---|---|---|---|---|
| B (ordem crescente), N registros | BST | ... | ... | ... |
| B (ordem crescente), N registros | AVL | ... | ... | ... |
| A (ordem original), N registros | BST | ... | ... | ... |
| A (ordem original), N registros | AVL | ... | ... | ... |

Depois da tabela, explique no vídeo (lembre-se de mostrar a tabela no vídeo), em poucas frases, **por que** os números ficaram assim (por exemplo, por que a diferença entre BST e AVL é pequena no Cenário X1 e grande no Cenário X2 nos seus dados). Os números precisam vir

da **execução real do seu programa**, e o professor poderá reproduzi-los com o seu arquivo de dados.

## 5. O Entregável Final

Entrega final até **22/10/26**, com os itens abaixo:

1. **Arquivo .c** comentado, contendo no início: nome do aluno, RA e o **link do vídeo** com permissão de acesso aberta ao professor.
2. **Arquivo de dados** (.txt ou .csv) usado na demonstração (mesmo da Etapa 1).
3. **Relatório em PDF (2 a 3 páginas):** problema e fonte real, por que árvore, tabela do experimento comparativo e o **Diário de Bordo**.
4. **Vídeo** de 15 a 25 minutos, conforme as regras e o roteiro da Seção 6.

## 6. Regras e Roteiro do Vídeo

### Regras de gravação

- **Duração:** mínimo de **15 minutos** e máximo de **25 minutos**. Fora da faixa há penalização.
- **Autoria:** seu **rosto deve aparecer durante todo o vídeo**, em um canto da tela.
- **Qualidade:** áudio claro e resolução que permita ler o código.
- **Sem cortes nos trechos de demonstração do código/programa ao vivo**. Esses trechos devem ser uma gravação contínua.

### Roteiro obrigatório

5. **Introdução:** apresente-se e conte o problema real em suas palavras, mostrando a fonte e explicando por que escolheu esse problema.
6. **Por que árvore?** explique a escolha da chave, compare com vetor/lista e diga o que a AVL resolve que a BST não resolve.
7. **Código:** explique a struct do nó, a alocação dinâmica, a rotação (a que você achar mais difícil), a remoção com dois filhos e onde está cada free.
8. **Demonstração ao vivo:** compile, rode com os seus dados reais, mostre o menu funcionando, mostre o experimento comparativo.
    - Execute o programa com **AddressSanitizer** (gcc -fsanitize=address -g) ou **valgrind**, evidenciando que não há vazamento de memória e nem acesso inválido de memória (Tarefa de pesquisa de conhecimento).
        gcc
        - gcc -fsanitize=address -g -Wall -o trabalho trabalho.c

        Valgrid:
        - gcc -g -Wall -o trabalho trabalho.c
        - valgrind --leak-check=full --show-leak-kinds=all ./trabalho
9. **Bug proposital:** introduza, ao vivo (sem cortes), um erro em alguma rotação ou na remoção de nós, mostre o sintoma no programa/execução e explique o raciocínio.
10. **Fechamento:** limitações da sua solução e o que mudaria se o volume de dados fosse 1000 vezes maior.

## 7. Diário de Bordo (no relatório)

Registre pelo menos **5 entradas datadas** com: o que você tentou, o **erro real** que encontrou (mensagem do compilador, segmentation fault, saída errada, árvore desbalanceada) e como o resolveu. Inclua ao menos **uma captura de tela de erro/situação real**.

## 🚀 8. Orientações para Envio

- Envie todos os arquivos pela plataforma da disciplina, em uma única entrega, até o prazo.
- Suba o vídeo em uma plataforma (Google Drive, YouTube não listado etc.) e coloque o **link de acesso público** no início do arquivo .c e no relatório. Teste o link em uma janela anônima antes de enviar.
- Atrasos e links sem permissão de acesso serão penalizados.